

UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA ESTADÍSTICA E INFORMÁTICA

CURSO: PROGRAMACIÓN NUMÉRICA

DOCENTE: ING. TORRES CRUZ FRED

ALUMNO: CHURATA MAMANI MILENA KELY



Trabajo 2 — Graficador de Funciones Lineales

Objetivo

Desarrollar un programa en Python que permita ingresar dos funciones lineales y graficarlas de manera visual en una ventana interactiva, mostrando los ejes, las líneas representativas de cada función y el punto de intersección si existe. Esta versión presenta mejoras gráficas, mayor validación de expresiones y una interfaz más elegante.

Descripción del problema

El usuario debe ingresar dos funciones lineales de la forma $y = ax + b$ (por ejemplo: $2x+3$ o $-x+5$). El programa graficará ambas líneas en un plano cartesiano, identificará si son paralelas o si se intersectan, y mostrará el punto de intersección cuando sea posible.

Restricciones

- 🎨 Solo se admiten funciones lineales simples con variable 'x'.
- 🎨 Las expresiones pueden contener multiplicaciones implícitas (como $2x$ o $-x$).
- 🎨 Rango de graficación: $x \in [-20, 20]$.
- 🎨 No se utilizan librerías externas, solo el módulo estándar tkinter.

Enfoque de la solución

1. Se validan las expresiones ingresadas para evitar caracteres no permitidos.
2. Se normalizan las funciones lineales, insertando el símbolo de multiplicación (*) donde sea necesario.
3. Se crea un plano cartesiano dentro de un lienzo gráfico (Canvas) de tkinter.
4. Se trazan las dos funciones con colores diferentes (rojo y azul).
5. Si las rectas se intersectan, se calcula el punto de intersección y se dibuja sobre el gráfico.
6. Se muestra la leyenda y las coordenadas del punto de cruce, si existe.



Código Fuente Python

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 def procesar_funcion_lineal(expresion):
5     expresion = expresion.replace(" ", "")
6     if expresion == "x":
7         return "1*x"
8     if expresion.startswith("x"):
45
46         pendiente = funcion_procesada.split('*x')[0]
47     if '+' in funcion_procesada:
48         partes = funcion_procesada.split('+')
49         intercepto = partes[1] if len(partes) > 1 else '0'
50     elif '-' in funcion_procesada and funcion_procesada.count(
51         partes = funcion_procesada.split('-')
52         intercepto = '-' + partes[2] if len(partes) > 2 else '
53     else:
54         intercepto = '0'
55
56     print(f"Función procesada: y = {pendiente}x + {intercepto}")
57     print(f"Pendiente: {pendiente}")
58     print(f"Intercepto con Y: {intercepto}")
59
60     plt.tight_layout()
61     plt.show()
62
63 if __name__ == "__main__":
64     graficar_funcion_lineal()
29     return
30
31 plt.figure(figsize=(12, 8))
32 plt.plot(x_valores, y_valores, 'red', linewidth=3, label=f
33
34 plt.axhline(y=0, color='black', linewidth=1, alpha=0.7)
35 plt.axvline(x=0, color='black', linewidth=1, alpha=0.7)
36
37 plt.grid(True, linestyle='--', alpha=0.5)
38 plt.xlim(-10, 10)
39 plt.ylim(-10, 10)
40
41 plt.title(f'Gráfico de la función: y = {entrada}', fontsize
42 plt.xlabel('Eje X', fontsize=12)
43 plt.ylabel('Eje Y', fontsize=12)
44 plt.legend(fontsize=12)
```

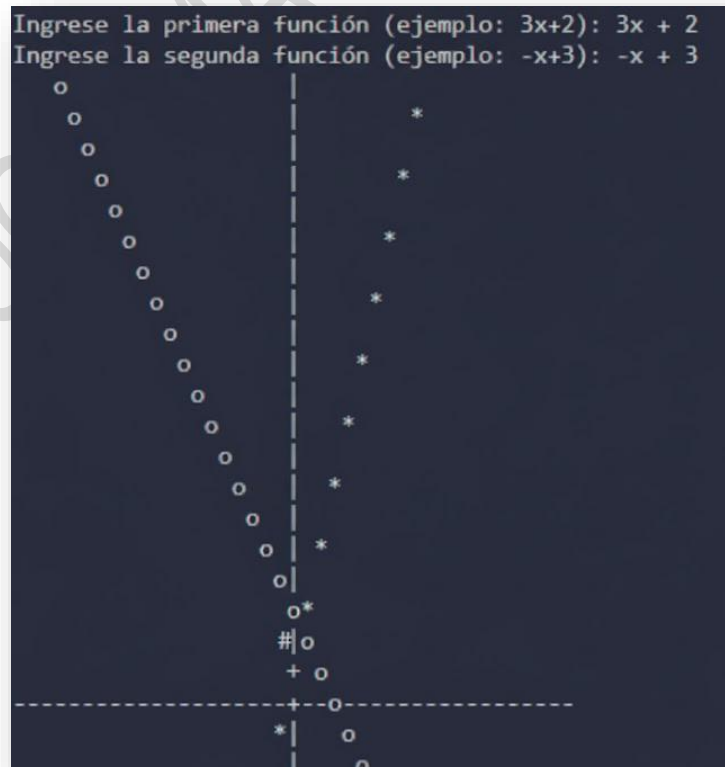
```

45 pendiente = funcion_procesada.split('*x')[0]
46 if '+' in funcion_procesada:
47     partes = funcion_procesada.split('+')
48     intercepto = partes[1] if len(partes) > 1 else '0'
49 elif '-' in funcion_procesada and funcion_procesada.count(
50     partes = funcion_procesada.split('-')
51     intercepto = '-' + partes[2] if len(partes) > 2 else '
52 else:
53     intercepto = '0'
54
55 print(f"Función procesada: y = {pendiente}x + {intercepto}")
56 print(f"Pendiente: {pendiente}")
57 print(f"Intercepto con Y: {intercepto}")
58
59 plt.tight_layout()
60 plt.show()
61
62 -
63 if __name__ == "__main__":
64     graficar_funcion_lineal()

```

Ejemplo de ejecución

Entrada: $f(x) = 3x + 2$



Conclusión

Esta versión mejorada ofrece una interfaz más intuitiva y una representación visual precisa de las funciones lineales. El uso de expresiones regulares para validar las entradas evita errores comunes y mejora la seguridad. Además, la inclusión de colores, leyendas y marcadores hace que el análisis gráfico sea más claro y profesional.

PROGRAMACION NUMERICA